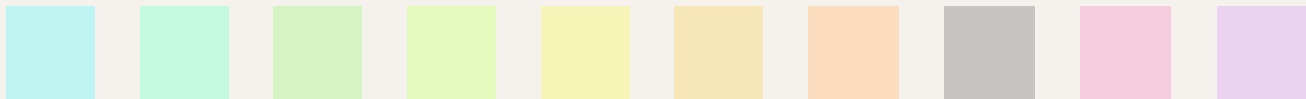




Server Side



מבוא לבניית מערכות ממוחשבות מבוססות אינטרנט הרצאה 6 – express



Express

Express.js is a popular and easy-to-use framework for building web servers with Node.js.

- It helps you create web and mobile apps quickly and easily.
- Express gives you useful tools to handle things like routes, requests, and responses.
- It's one of the most widely used frameworks in the Node.js world -many websites use it behind the scenes.
- You don't need to build your own server from scratch. Express handles that for you!
- If you know a bit of JavaScript, Express makes building websites and APIs much simpler.

Getting started

Assuming you've already installed Node.js, in your command line prompt:

1. create a directory to hold your application

```
$ mkdir myapp
```

2. make that your working directory

```
$ cd myapp
```

Getting started

3. Use the `npm init` command to create a `package.json` file for your application.

```
$ npm init  
OR  
$ npm init -y
```

This command prompts you for a number of things, such as the name and version of your application.

The name of the
npm package
(=name of project)

The name of the
main .js file

Your name

The .json file preview

```
$ npm init
```

This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.

package name: (test-to-selete)

version: (1.0.0)

description:

entry point: (index.js)

test command:

git repository:

keywords:

author:

license: (ISC)

About to write to C:\Users\itzik\OneDrive\מחלונות\test to selete\package.json:

```
{  
  "name": "test-to-selete",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "author": "",  
  "license": "ISC"  
}
```

Is this OK? (yes) ☐

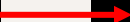
What is a package.json file?

- A package.json file is a metadata file commonly used in Node.js projects.
- It plays a crucial role in managing a project's dependencies, scripts, and various other settings.
- key aspects of a package.json file:
 - **Metadata-** The file includes metadata about the project, such as the project name, version, description, author, and license information.
 - **Dependencies-** package.json file specify the project's dependencies- external libraries or modules that the project relies on to run successfully.
 - **Scripts**
 - **Versioning**
 - **Etc.**

```
$ npm i
```

The Package.Json

To allow working
with "import"



```
{
  "name": "my-project",
  "version": "1.0.0",
  "description": "A description of my project",
  "main": "index.js",
  "Type": "Module",
  "scripts": {
    "start": "node index.js",
    "test": "mocha tests/*.js"
  },
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "mocha": "^8.5.1"
  },
  "author": "Your Name",
  "license": "MIT"
}
```

What is JSON?

- JSON stands for JavaScript Object Notation
- It's a simple and lightweight way to store and send data
- Commonly used to send data between a server and a web page
- JSON is easy to read and write – it looks like regular JavaScript objects
- It's self-explanatory – you can often understand it just by looking at it

Summary: Why JSON and package.json Matter

- JSON is the language of data in web development
 - It's how data is structured, sent, and understood
- package.json is like your project's "ID card" and toolbox
 - It describes your app and tracks everything it needs to run
- When you install packages or share your code, package.json keeps it organized, consistent, and easy to manage

Getting started

4. Now install Express in the myapp directory :

```
$ npm i express
```

- Let's check our project's folder and see what's in it



express

Creating a Local Server

Creating a Local Server

5. In the myapp directory, create a file named app.js and copy in the code:

```
import express from 'express';
const app = express();
const port = 8080;

app.get('/', (req, res) => { //for home page
  res.send('Hello World!');
})

app.listen(port, () => {
  console.log(`listening on port ${port}`);
})
```

Creating a Local Server

- This app starts a server and listens on port 8080 for connections.
- The app responds with “Hello World!” for requests to the root URL (/) or route.

What is a Port?

- A port is like a doorway to your computer that helps it talk to the outside world.
- Your computer can run many programs at once (e.g., a web server, email app, chat app).
- Each program listens on a different port so that messages go to the right place.
- Web servers usually run on:
 - Port 80 for normal websites (HTTP)
 - Port 443 for secure websites (HTTPS)
 - Port 3000, 5000, or 8080 are often used for local development
- So when you run an Express app on port 3000, it means:
"Start a server, and listen for incoming connections through door number 3000."

Before we run the server- Nodemon!

- To run the server, we use node + filename.js
- With every change made in the filename.js, we need to restart the server.
- Using nodemon to run the server:
 - Type `npm i -g nodemon`
 - nodemon will restart the server every time you press ctrl+S in your server file.
- Let's:
 1. Run your server on the command line using `nodemon+filename.js`
 2. Check port 8080 on your browser



express

HTTP Requests

Basic HTTP requests

Get

"Ask the server for information (like loading a webpage)."

Post

"Send new data to the server (like submitting a form)."

Put

"Send all data to update something completely."
(like replacing a document in the computer- you send the entire resource again)

Patch

"Send only the changes to update something."
(like editing a document on the computer- you send only the parts you need to change)

Delete

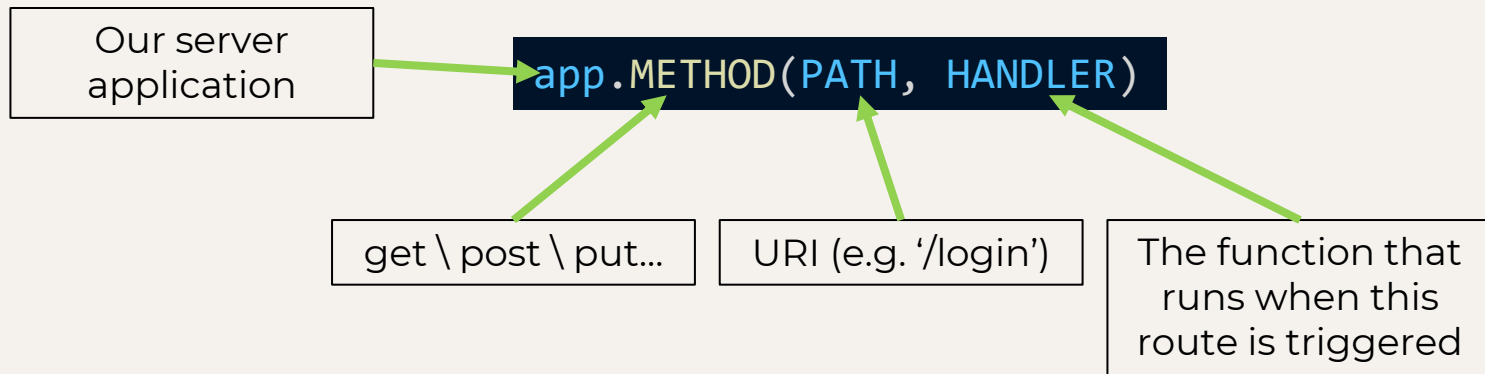
"Tell the server to remove something."

express

Routing

Routing in Express.js

- Routing means deciding how your server responds when someone visits a specific URL using a certain HTTP method (like GET or POST).



Routing

```
app.METHOD(PATH, HANDLER)
```

- key aspects of routes:
 - **Path (URI)-**
 - The URL where the route "listens" (e.g., /users).
 - When a user visits this URL, the server responds.
 - **HTTP method-**
 - Tells the server what kind of action is happening:
 - GET to fetch data, POST to send data, etc.
 - **Handler-**
 - The function that runs when the server gets a request that matches both the path and the method.

Routing- Example

```
import express from 'express';
const app = express();
const port=8080;

// Define a route for the homepage ('localhost:8080/')
app.get('/', (req, res) => {
  res.send('Hello, World!');
});

// Define a route for ('localhost:8080/users')
app.get('/users', (req, res) => {
  res.send('List of users');
});

// Start the server on port 8080
app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

Routing Methods

Common Routing Methods in Express:

1. `app.get(path, handler)`

→ Handles reading data (like showing a webpage or list)

2. `app.post(path, handler)`

→ Handles sending or saving data (like submitting a form)

3. `app.use(middleware)`

→ Adds middleware (functions that run before routes)

4. `res.render(view)`

→ Sends a view/template (HTML) to the browser

Full list [here](#)

Route paths (1)

```
app.METHOD(PATH, HANDLER)
```

- A route path is the URL (like /login, /about, /users/:id) where your app listens for a request.
- Combined with a request method (like GET, POST), it defines what happens at that **endpoint**.

```
app.get('/users', function (req, res) {  
  res.send('user list');  
});
```

- This means: When a user goes to /about using a GET request- the server sends back “Welcome to the About page”

Route paths (2)

Route Paths Can Be:

- Exact strings → /login
- Patterned paths → /users/:userId
- Regex (advanced) → /^\/a/ matches any path starting with /a

Route Parameters

- Route parameters are named URL segments that are used to capture the values specified at their position in the URL.
- The captured values are populated in the **req.params** object, with the name of the route parameter specified in the path as their respective keys.

Route path: /users/:userId/books/:bookId

Request URL: http://localhost:3000/users/34/books/8989

req.params: { "userId": "34", "bookId": "8989" }

Route Parameters

- To define routes with route parameters, simply specify the route parameters in the path of the route as shown below.

```
app.get('/users/:userId/books/:bookId', function (req, res) {  
  res.send(req.params)  
})
```

Route Parameters

- Since the hyphen (-) and the dot (.) are interpreted literally, they can be used along with route parameters for useful purposes.

Route path: /flights/:from-:to

Request URL: http://localhost:3000/flights/LAX-SFO

req.params: { "from": "LAX", "to": "SFO" }

Route path: /plantae/:genus.:species

Request URL: http://localhost:3000/plantae/Prunus.persica

req.params: { "genus": "Prunus", "species": "persica" }

Response Methods

- The methods on the response object (res) can send a response to the client and terminate the request-response cycle.
- If none of the methods are called from a route handler, the client request will be left hanging.

Response methods

Method	Description
<u>res.download()</u>	Prompt a file to be downloaded.
<u>res.end()</u>	End the response process.
<u>res.json()</u>	Send a JSON response.
<u>res.redirect()</u>	Redirect a request.
<u>res.render()</u>	Render a view template.
<u>res.send()</u>	Send a response of various types.
<u>res.sendFile()</u>	Send a file as an octet stream.
<u>res.sendStatus()</u>	Set the response status code and send its string representation as the response body.

express

Path Module

Path module

- The Path module provides a way of working with directories and file paths.
- The syntax for including the Path module in your application:

```
var path = require('path');  
//OR  
import path from 'path';
```

Path Methods

Method	Description
<code>basename()</code>	Returns the last part of a path
<code>dirname()</code>	Returns the directories of a path
<code>extname()</code>	Returns the file extension of a path
<code>format()</code>	Formats a path object into a path string
<code>isAbsolute()</code>	Returns true if a path is an absolute path, otherwise false
<code>join()</code>	Joins the specified paths into one
<code>normalize()</code>	Normalizes the specified path
<code>parse()</code>	Formats a path string into a path object
<code>posix</code>	Returns an object containing POSIX specific properties and methods
<code>relative()</code>	Returns the relative path from one specified path to another specified path
<code>resolve()</code>	Resolves the specified paths into an absolute path

__dirname

- The directory of the current module where the code is executed.
 - When using common-js the __dirname is automatically defined.
 - When using ES5 the __dirname needs to be defined.

```
import { fileURLToPath } from "url";  
import path from 'path'; // Add this line to import the path module  
  
// when using ES5:  
console.log(import.meta.url);  
file:///C:/Users/itzik/OneDrive/%D7%A9%D7%95%D7%9C%D7%97%D7%9F%20%D7%94%D7%A2%D7%91%D7%95%D7%93%D7%94/%D7%A2%D7%91%D7%95%D7%93%D7%94%20%D7%90%D7%95%D7%A0%D7%99%D7%91%D7%A8%D7%A1%D7%99%D7%98%D7%AA%20%D7%91%D7%9F%20%D7%92%D7%95%D7%A8%D7%99%D7%95%D7%9F/WE
```

URL Formatted Path. The file:// indicates a file resource on the local server. The rest is a URL-encoded. For example- %D7%A9 is the letter ש in Hebrew

```
console.log(fileURLToPath(import.meta.url));
```

The same URL Formatted Path in a standard windows file system path

```
console.log(dirname(fileURLToPath(import.meta.url)));  
C:\Users\itzik\OneDrive\שולחן העבודה\עבודה אוניברסיטת בן גוריון\WEB\הרצאות-LAPTOP-F5BD2N7D-LAPTOP-F5BD2N7D\איציק\lectures examples\Express  
  
const __dirname = dirname(fileURLToPath(import.meta.url));
```

path.join() Method

Definition and Usage

- The path.join() method joins the specified path segments into one path.
- The specified path segments must be strings, separated by comma.
- `path.join(__dirname, '/public/HTMLFile.html');`



express

Static Files

Serving static files in Express

- `express.static` built-in middleware function is used to serve static files such as images, CSS files, and JavaScript files **without the need for a specific route handler**.

```
express.static(root, [options])
```

- The `root` argument specifies the root directory from which to serve static assets
- `Options` is an object specifies which files can be sent using `express.static` (e.g. only html, allow .hidden files, only files with specific extension etc.)

Serving static files in Express

- For example, use the following code to serve images, CSS files, and JavaScript files in a directory named 'public':

```
app.use(express.static('public'))
```

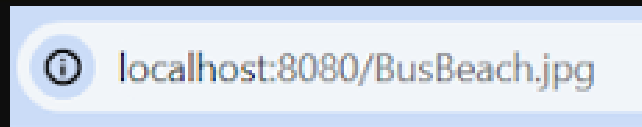
- Now, the files that are in the public directory can be loaded.

EXPRESS

- node_modules
- public
 - BusBeach.jpg
 - index.html
 - style.css
- An Example PDF.pdf
- app.js
- index.html
- package-lock.json
- package.json



```
app.use(express.static('public'));
```



EXPRESS

- > node_modules
- public
 - BusBeach.jpg
 - index.html
 - style.css
- An Example PDF.pdf
- app.js
- index.html
- package-lock.json
- package.json



```
app.use(express.static('public'));
```




express

Middlewares

Middleware

- A middleware is a function that have access to the request object (req), the response object (res) and the next middleware function in the application's request-response cycle.
- Middlewares have 4 main categories:
 - Preprocessing
 - Authentication
 - Error handling
 - logging

Body-Parser

- One of the most common express middleware.
- Used to parse the body of the request in various formats- URL-Encoded, JSON, etc.
- **Particularly useful when dealing with data sent from an html form.**

```
import body-parser from 'body-parser';
```

Band name generator using body-parser

```
<body>
  <h1>Band Name Generator</h1>
  <form action="/submit" method="POST"> <!--the
action=submit sends the content to the submit page-->
    <label for="veg">Favorite Vegetable:</label>
    <input type="text" name="veg" required>
    <label for="pet">Pet Name:</label>
    <input type="text" name="pet" required>
    <input type="submit" value="Submit">
  </form>
</body>
```

Band name generator using body-parser



A screenshot of a web browser window. The address bar shows 'localhost:3000'. The page title is 'Band Name Generator'. Below the title, there is a form with two input fields: 'Favorite vegetable:' with the value 'bellpepper' and 'Pet Name:' with the value 'Miguel'. A 'Submit' button is located to the right of the 'Pet Name' field.

localhost:3000

Band Name Generator

Favorite vegetable: Pet Name:

Band name generator using body-parser



A screenshot of a web browser window. The address bar shows 'localhost:3000'. The page title is 'Band Name Generator'. Below the title, there are two input fields: 'Favorite vegetable:' with the value 'bellpepper' and 'Pet Name:' with the value 'Miguel'. To the right of these fields is a 'Submit' button.

localhost:3000

Band Name Generator

Favorite vegetable: Pet Name:

Submit



A screenshot of a web browser window. The address bar shows 'localhost:3000/submit'. The page content displays 'Your band name is:' followed by the generated band name 'bellpepperMiguel' and a peace sign emoji.

localhost:3000/submit

Your band name is:

bellpepperMiguel🕊



express

Database Integration

Database integration

- Adding the capability to connect databases to Express apps is just a matter of loading an appropriate Node.js driver for the database in your app.
- This section briefly explains how to add and use mySql Node.js module for database systems in your Express app

```
$ npm install mysql2
```




express

Database Integration

step by step example

Step 1: Create Node.js application

- Open terminal/console, then create a folder for our application:

```
mkdir DatabaseIntegration  
cd DatabaseIntegration
```

- Initialize the Node.js application with a package.json file:

```
npm init -y
```

- Next, we need to install necessary modules: express, mysql and body-parser. Run the command:

```
npm i express mysql2 body-parser
```

Step 2: Setup Express web server

- In the root folder, we create a new file named index.js:

```
const express = require("express");
const bodyParser = require("body-parser");
const app = express();
const port = 3000;

// parse requests of content-type: application/json
app.use(bodyParser.json());
// parse requests of content-type: application/x-www-form-urlencoded
app.use(bodyParser.urlencoded({ extended: true }));

// set port, listen for requests
app.listen(port, () => {
  console.log(`Server is running on port ${port}.`);
});
```

Open MySql command line

Before connecting Node.js Application with MySQL, we need to create a table.

1. Open Command Prompt.
2. Navigate to the bin folder.

For example: `cd C:\Program Files\MySQL\MySQL Server 8.0\bin`

1. Run the **mysql -u root -p** command.
2. Enter the password.

Step 3: Create MySQL table

- Run the SQL script (in MySQL command line or in MySQL workbench) below to create customers' table:

```
use mysql;
```

```
CREATE TABLE IF NOT EXISTS `customers` (id int (11) NOT NULL PRIMARY KEY AUTO_INCREMENT, email varchar(255) NOT NULL, name varchar(255) NOT NULL) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

```
INSERT INTO customers (name, email) VALUES ("bob dylan", "bobyd@gmail.com");
```

Step 4: Configure & Connect to MySQL DB

- create db.config.js file inside the root folder with:

```
module.exports = {  
  HOST: "localhost",  
  USER: "root",  
  PASSWORD: "*****", //the password to your MySql Databases  
  DB: "mysql"  
};
```

Step 4: Configure & Connect to MySQL DB

- Now create a database connection that uses configuration above.
- The file for connection is *db.js* saved in the root folder as well.

```
const mysql = require("mysql2");
const dbConfig = require("../db.config.js");

// Create a connection to the database
const connection = mysql.createConnection({
  host: dbConfig.HOST,
  user: dbConfig.USER,
  password: dbConfig.PASSWORD,
  database: dbConfig.DB
});

// open the MySQL connection
connection.connect(error => {
  if (error) throw error;
  console.log("Successfully connected to the database.");
});

module.exports = connection;
```

Step 5: Define routes for CRUD operations

- What is CRUD operations?
 - Create
 - Read
 - Update
 - Delete

Step 5: Define routes for CRUD operations

- In index.js define DB connection and use it to write CRUD functions:
 1. get all Customers
 2. create a new Customer
 3. find a Customer by name
 4. update a Customer by name
 5. remove a Customer
 6. remove all Customers
- Lets start with “get all Customers”

“get all Customers” in one block in index.js

- In index.js add a route for “get all Customers”

```
var sql = require('./db')

// Create a route for getting all customers
app.get("/customers", function(req, res){
  sql.query("SELECT * FROM customers", (err, mysqlres) => {
    if (err) {
      console.log("error: ", err);
      res.status(400).send( "error in getting all customers: " + err);
      return;
    }

    console.log("got all customers...");
    res.send(mysqlres);
    return;
  });
});
```

“Create new customer” split into 2 files

- Create a new file named `CRUD_functions.js` (on root folder)
- In this file define the following function:

```
const sql = require("./db.js");
const createNewCustomer = function(req,res){
  // Validate request
  if (!req.body) {
    res.status(400).send({message: "Content can not be empty!"});
    return;
  }
  const newCustomer = {
    "email": req.body.email,
    "name": req.body.name
  };
  sql.query("INSERT INTO customers SET ?", newCustomer, (err, mysqlres) => {
    if (err) {
      console.log("error: ", err);
      res.status(400).send( "error in creating customer: " + err);
      return;
    }
    console.log("created customer: ", { id: mysqlres.insertId, ...newCustomer });
    res.send("new customer created successfully");
    return;
  });
};
module.exports = {createNewCustomer};
```

“Create new customer” split into 2 files

- Inside index.js define a route using the handler as the function from CRUD_functions.js:
1. At the top of the file, right after the app.use(bodyParser) link the CRUD_functions.js by adding:

```
const CRUD_operations = require("../CRUD_functions.js");
```

2. Then define route for post request at path /newCustomer:

```
// Create a new Customer  
app.post("/CreateNewCustomer", CRUD_operations.createNewCustomer);
```

“Create new customer” client side

- Create an HTML file in the root folder called “Create_new_customer.html”

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Form connected to DB</title>
</head>
<body>
  <form action="/CreateNewCustomer" method="POST">
    <label for="name">Your Name</label>
    <input type="text" id="name" name="name"><br>
    <label for="email">Your email</label>
    <input type="text" id="email" name="email"><br>
    <input type="submit" value="Submit">

  </form>
</body>
</html>
```

Serve html form as index

- Meanwhile, save the form as the index file for path “/” in the root folder
- Serve this file for this path by creating the proper route in index.js file using res.sendFile().
- To index.js add at the top:

```
var path = require('path');
```

- And add the route for the get request for ‘/’:

```
app.get('/', (req,res) => {  
  res.sendFile(path.join(__dirname, “/Create_new_customer.html”))  
});
```

Check yourself

- Open your browser to <http://localhost:8080/>
- Submit a form
- You should see a message:
“new customer created successfully”

“find a Customer by name”

- In CRUD_functions.js define the function for finding a customer by name
- We will use the method “get”

```
const FindCustomer = (req,res)=>{
  // check if body is empty
  if (!req.body) {
    res.status(400).send("content can not be empty");
    return;
  }
  //if body not empty - create new customer
  const Customer = req.query.FindName;
  console.log(Customer);
  //insert query
  sql.query("SELECT * FROM customers where name like ?", Customer + "%", (err, results, fields)=>{
    if (err) {
      console.log("error is: " + err);
      res.status(400).send( "error in finding customer " + err);
      return;
    }
    // if not query error
    console.log("new customer created ", createNewCustomer);
    res.send(results)
    //res.send({message: "new customer created successfully"});
    return;
  });
};

module.exports = {FindCustomer};
```

“find a Customer by name”

- In index.js define the appropriate route:

```
// get customer by name  
app.get("/FindCustomersByName", CRUD_operations.FindCustomer);
```

“find a Customer by name” – client side

- Go to:
<http://localhost:8080/FindCustomersByName?FindName=>
- At the end add the name you are looking for (that's the query parameters)